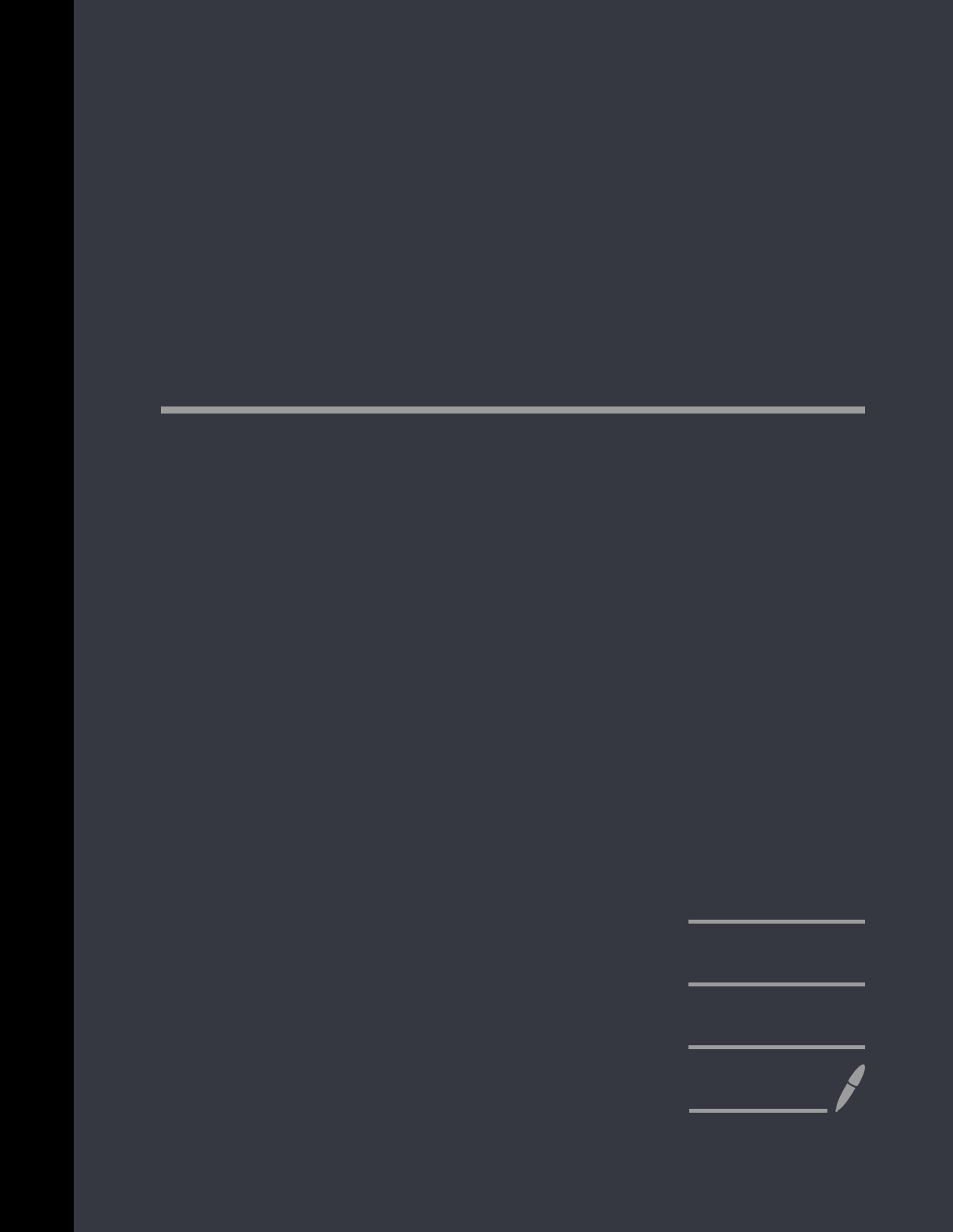
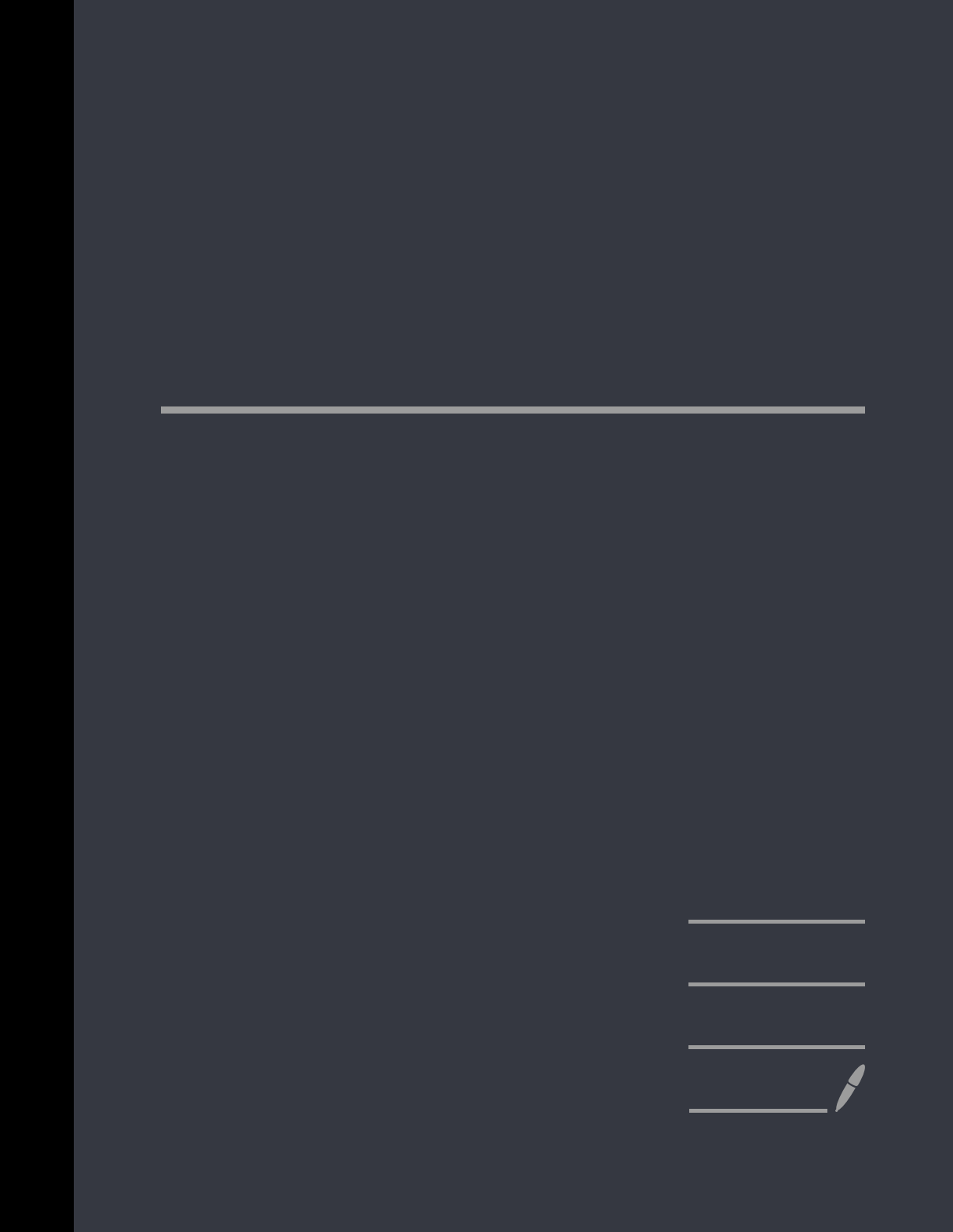
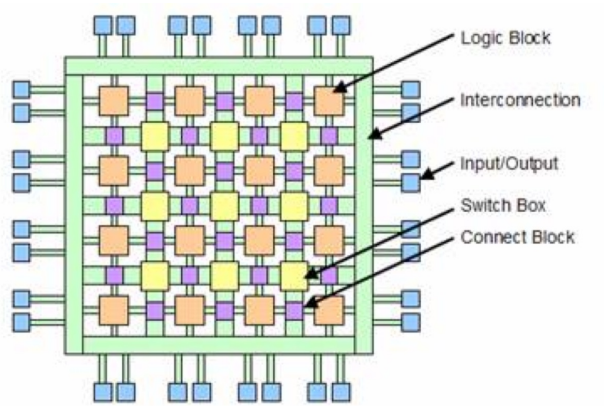


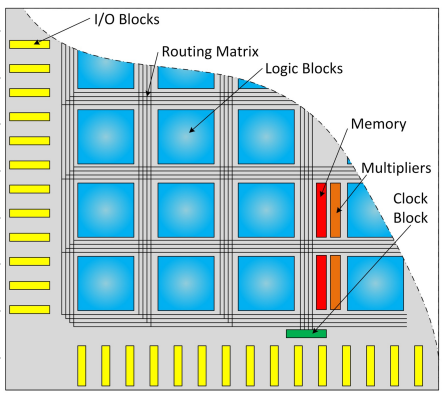


Aula 1

FPGA → Field Programable Gate Array

São vários blocos lógicos interligados e que podem ser reprogramados





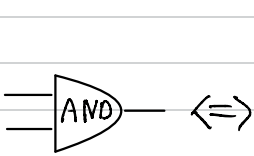
Blocos lógicos

- LUTs
- Entrada e saída (I/O)
- Flip Flops

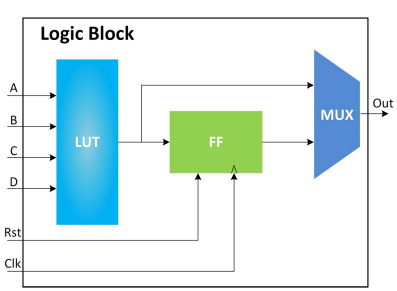
→ Logic cells / Logic block

Although the exact architecture of LCs might vary slightly across different FPGAs, their general structure is quite similar, as shown in the figure below:

LUT → Look Up Table



LUT	
0 0	0
0 1	0
1 0	0
1 1	1



Componentes da FPGA

- Blocos lógicos configuráveis
  - Constituídos por LUTs, flip-flops, …
- Blocos de entrada/saída (I/O)
  - Buffers, que funcionam como um pino bidirecional de entrada e saída da FPGA, com a possibilidade de registar sinais
- Blocos de comutação programáveis
  - Trilhas/linhas utilizadas para conectar os blocos lógicos e blocos I/O. O processo de escolha das interligações é chamado de encaminhamento

Um circuito é constituído por vários blocos lógicos, blocos de comutação/interligação e blocos de entrada/saída

Podemos dar flash de uma configuração na FPGA mas assim que a desligarmos ela esquece tudo.

Aula 2 - 21/02

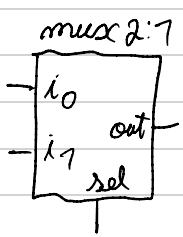
- VHDL
  (HDL) → Parecido com linguagem de programação
  mas não é!!

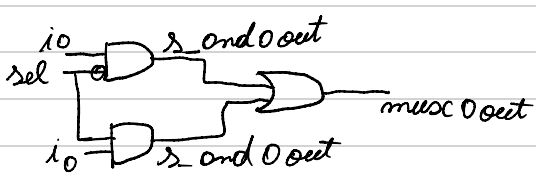
- Abstrações do VHDL
  - Entidades
  - ...
- Tipos de dados em VHDL
  - Só vamos falar de um tipo hoje. → Std_Logic

A primeira linha de qualquer documento vai sempre ser "library IEEE".

exemplo: library IEEE
use IEEE.std_logic_1164.all

Exemplo em VHDL (MUX 2→1)





VHDL não é case sensitive

a = A

Código:

```
library IEEE
use IEEE.STD_LOGIC_1164.all;
```

} biblioteca

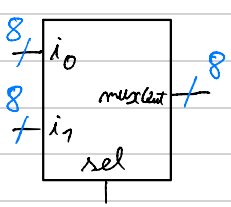
```
entity Musc2_1 is
    port ( sel    : in std_logic;
           input0 : in std_logic;
           input1 : in std_logic;
           muxCout: out std_logic;
    end Musc2_1
```

} Entidade

```
architecture Beh of Musc2_1 is
    signal S_and0out, S_and1Cout : std_logic;
    Begin
        S_and0Cout <= input0 and (not sel);
        S_and1Cout <= input1 and sel;
        muxCout <= S_and0Cout or and1Cout
end Beh;
```

} Arquitetura

Musc de 8 bits (IDK ¯_(ツ)_/¯)



...

entity Mux 2_1 is

port (input$_0$, input$_1$: in std_logic_vector (7 downto 0)

mux out : out std_logic_vector (7 downto 0)

...

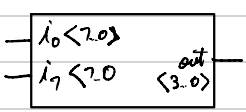
Comentários: /* ~~~~

~~~~

~~~~

*/

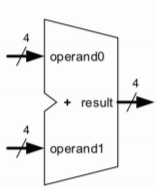
VHDL decodificador



Exemplo de Circuito Aritmético – Somador Binário de 4 bits

- *Entity*

```
entity Adder4 is
    port(operand0 : in  std_logic_vector(3 downto 0);
         operand1 : in  std_logic_vector(3 downto 0);
         result   : out std_logic_vector(3 downto 0));
end Adder4;
```



3

architecture B of Adder 4

- Este n tem carry
- O somador a seguir já tem.

. . .

VHDL suporta signed (+ −) e unsigned (+)

Modelação Comportamental do Somador (sem Carry In/Out)

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD.all;

entity Adder4 is
    port(operand0 : in  std_logic_vector(3 downto 0);
         operand1 : in  std_logic_vector(3 downto 0);
         result   : out std_logic_vector(3 downto 0));
end Adder4;

architecture Behavioral of Adder4 is
begin
    result <= std_logic_vector(unsigned(operand0) +
                               unsigned(operand1));
end Behavioral;
```

lib

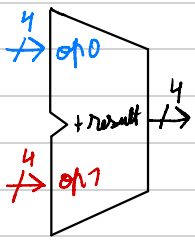
portas do adder

função

Recomendação: usar sempre `std_logic(_vector)` nos portos da entidade

Conversão entre tipos: `std_logic_vector(…)` e `unsigned(…)`

Para que serve a biblioteca `IEEE.NUMERIC_STD`?



6

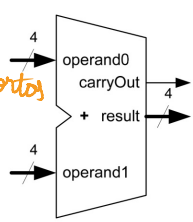
Somador com saída carry out

Adição da Saída Carry Out

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD.all;

entity Adder4 is
    port(operand0 : in  std_logic_vector(3 downto 0);
         operand1 : in  std_logic_vector(3 downto 0);
         result   : out std_logic_vector(3 downto 0);
         carryOut : out std_logic);
end Adder4;

architecture Behavioral of Adder4 is

    signal s_operand0, s_operand1, s_result : unsigned(4 downto 0);

begin
    s_operand0 <= '0' & unsigned(operand0);
    s_operand1 <= '0' & unsigned(operand1);
    s_result   <= s_operand0 + s_operand1;
    result     <= std_logic_vector(s_result(3 downto 0));
    carryOut   <= std_logic(s_result(4)); -- std_logic(...) opcional
end Behavioral;
```

lib

portas



5 saídas

Concatenar

Operador "&" – concatenação / justaposição

Definição formal dos tipos **unsigned** e **signed** de VHDL em **IEEE.NUMERIC_STD**:

```
type unsigned is array (natural range <> ) of std_logic;
type signed is array (natural range <> ) of std_logic;
```

TPC: O que seria necessário fazer para incluir também um "carryIn"?

7

O carry out deve ser quando chegamos à máxima capacidade de soma do adder, o ("e vai um") passa para o próximo adder.

ex: 1 (carry) 1 1 0 0 + 1 0 1 0 = 0 1 1 0

C. aux: 1+1 = 10

Aula ... sei lá leem... 4 ??

Circuitos sequenciais em VHDL

- Latch D
- Flip-Flop D

Funcionamento da latch D

D Latch (Structure and Operation)

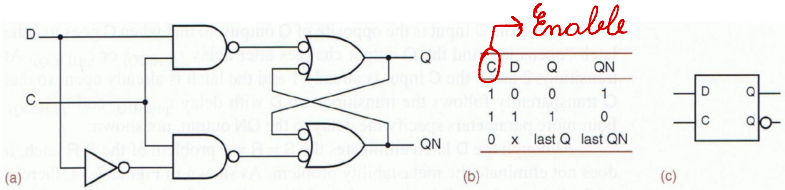


Figure 7-12 D latch: (a) circuit design using NAND gates; (b) function table; (c) logic symbol.

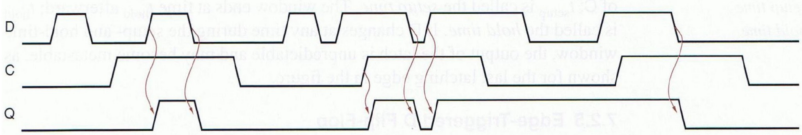
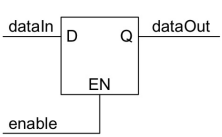


Figure 7-13 Functional behavior of a D latch for various inputs.

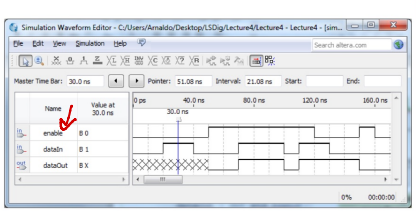
Implementação da latch D em VHDL

Módulo Sequencial Trivial – Latch D

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity LatchD is
   port(enable  : in  std_logic;
        dataIn  : in  std_logic;
        dataOut : out std_logic);
end LatchD;

architecture Behav of LatchD is
begin
   process(enable, dataIn)
   begin
     if (enable = '1') then
       dataOut <= dataIn;
     end if;
   end process;
end Behav;
```



A saída "segue" a entrada quando `enable='1'`



Porque razão o sinal `dataOut` surge inicialmente como "XXXXXXXX"? É relevante? Como resolver?

Pq o enable está a 0.

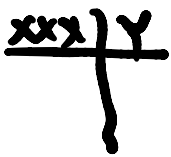
Aula 6

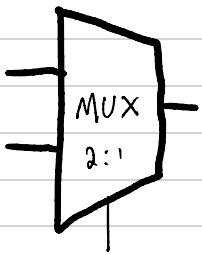
Exercícios

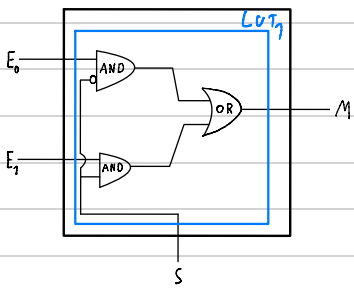
What is the minimum number of 3:1 LUTs required to implement a 2-bit 2:1 multiplexer?

Select one:

- 3
- 4
- 2
- 1 (selected)





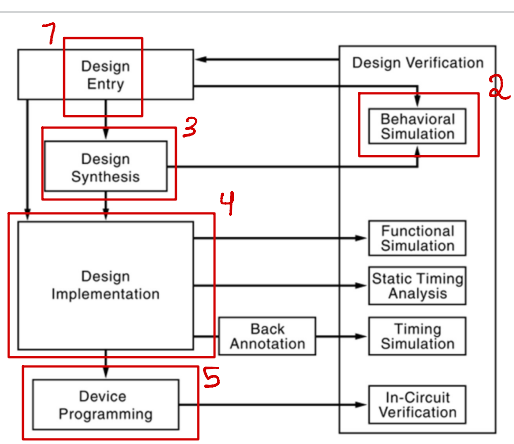


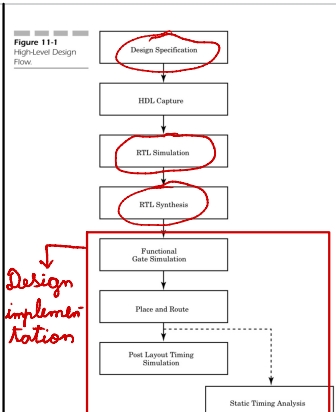
A digital system design flow based on FPGA consists in the following sequence of operations (in order):

Select one:

- implementation, device programming, modelling, simulation, synthesis, test and debug
- modelling, simulation, synthesis, implementation, device programming, test and debug (selected)
- simulation, modelling, implementation, synthesis, device programming, test and debug
- device programming, test and debug, modelling, implementation, synthesis, simulation

Design entry / specification ou modeling
↓
simulation
↓
synthesis
↓
Implementation
↓
device programming
↓
Test and debug



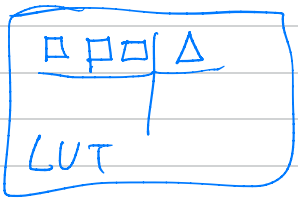


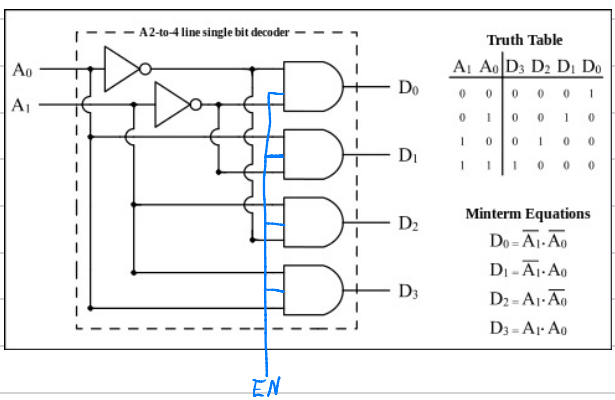
What is the minimum number of 3:1 LUTs required to implement a 2:4 binary decoder?

Select one:

- ○ 2
- ○ 1
- ◉ 4
- ○ 8

nº de luts = nº de saídas





The synthesis of the following code results in which of the following components:

```
process(inputs)
begin
   if (inputs = "00") then
      res <= "0001";
   elsif (inputs = "01") then
      res <= "0010";
   elsif (inputs = "10") then
      res <= "0100";
   else
      res <= "1000";
   end if;
end process;
```

tá certo

Select one:

- ○ A 4:1 multiplexer → nem existe (tinha que ser 4:2)
- ○ A binary 2:4 encoder
- ◉ A binary 2:4 decoder
- ○ None of the remaining answers

Clear my choice

Time left 0:01:05

The FPGA in the DE2-115 4 kit is a programmable logical device:

Select one:

- constituted by an homogeneous array composed only by configurable combinational logic cells
- constituted by an homogeneous array composed only by configurable input/output blocks and fixed routing resources
- constituted by an array of configurable logic cells, input/output blocks and routing resources
- configurable just once at the manufacturing time → Literalmente cada vez que ligar lol

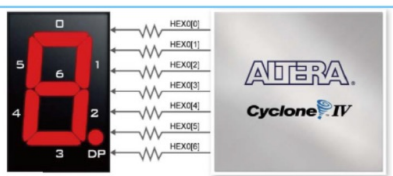
Clear my choice

tá certo!

Next page

Jump to...

Time left 0:08:31



Consider connections between the FPGA and the HEX0 display of the kit DE2-115 illustrated in the figure. To switch on a certain segment, it is necessary apply the logic value '0' to the respective FPGA output. Complete the following VHDL code to implement a circuit that displays on the HEX0 display the decimal digit 1 when sw(0)='0' and the decimal digit 2 when sw(0) = '1' **Do not use spaces in your responses.**

```
library IEEE ;
use IEEE.STD_LOGIC_1164.all;

entity display is
   port( sw : in std_logic_vector(0 downto 0);
         hex0: out std_logic_vector(6 downto 0));
end display;

architecture Behavioral of display is
begin

        hex0 <= "" when (sw(0) = '0') else
                "0100100";

end Behavioral ;
```

→ cada segmento

1111001 → 1 é desligado e 0 é ligado

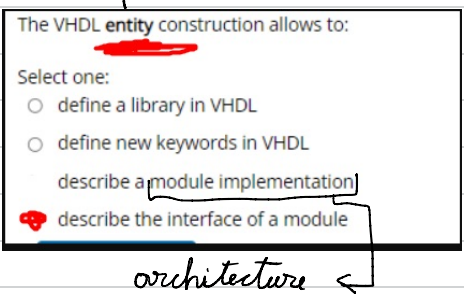
Time left 0:19:54

Consider connections between the FPGA and the HEX0 display of the kit DE2-115 illustrated in the figure. To switch on a certain segment, it is necessary apply the logic value '0' to the respective FPGA output. Complete the following VHDL code to implement a circuit that displays on the HEX0 display the decimal digit 1 when sw(0)='0' and the decimal digit 2 when sw(0) = '1' **Do not use spaces in your responses.**

```
library IEEE ;
use IEEE.STD_LOGIC_1164.all;

entity display is
   port( sw : in std_logic_vector(0 downto 0);
         hex0: out std_logic_vector(6 downto 0));
end display;

architecture Behavioral of display is
begin

        hex0 <= "" when (sw(0) = '0') else
                "0100100";

end Behavioral ;
```

Digit 1: "1111001"
Digit 2: "0100100"
Digit 3: "0110000"
Digit 4: "0011001"

→ na entity defines as portas



Entity

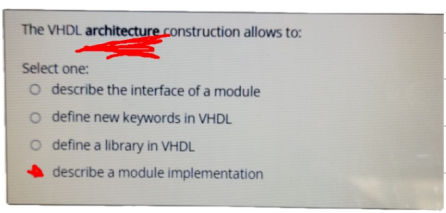
A VHDL entity specifies the name of the entity, the ports of the entity, and entity-related information. All designs are created using one or more entities.

Let's take a look at a simple entity example:

```
ENTITY mux    IS
  PORT ( a, b, c, d : IN BIT;
         s0, s1 : IN BIT;
         x,  : OUT BIT);
END mux;
```

The entity describes the interface to the VHDL model. The architecture describes the underlying functionality of the entity and contains the statements that model the behavior of the entity. An architecture is always related to an entity and describes the behavior of that entity. An architecture for the counter device described earlier would look like this:

```
ARCHITECTURE dataflow OF mux IS
  SIGNAL select : INTEGER;
BEGIN
  select <= 0 WHEN s0 = '0' AND s1 = '0' ELSE
            1 WHEN s0 = '1' AND s1 = '0' ELSE
            2 WHEN s0 = '0' AND s1 = '1' ELSE
            3;

  x <= a AFTER 0.5 NS WHEN select = 0 ELSE
       b AFTER 0.5 NS WHEN select = 1 ELSE
       c AFTER 0.5 NS WHEN select = 2 ELSE
       d AFTER 0.5 NS;

END dataflow;
```

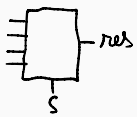


The keyword **ARCHITECTURE** signifies that this statement describes an architecture for an entity. The architecture name is **dataflow**. The entity the architecture is describing is called **mux**.

n 2
ed
l out of
n

The synthesis of the following code results in which of the following components:

```
res     <=      in0 when (s = "00") else
                in1 when (s = "01") else
                in2 when (s = "10") else
                in3;
```



Select one:

- A 4:1 multiplexer
- A binary 2:4 encoder
- A binary 2:4 decoder
- None of the remaining answers

Pergunta **5**
Por responder
Nota: 4,0
Marcar pergunta

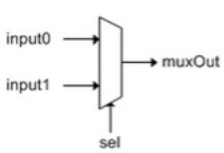
A síntese do seguinte código resulta em qual dos seguintes componentes:

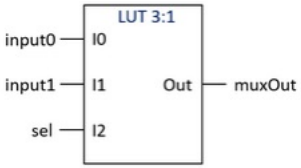
```
outputs <=      "0001" when (inputs = "00") else
                "0010" when (inputs = "01") else
                "0100" when (inputs = "10") else
                "1000";
```

Selecione uma opção de resposta:

- Um descodificador binário de 2:4
- Um codificador binário de 4:2
- Nenhuma das respostas restantes
- Um multiplexer de 4:1

Qual deve ser o conteúdo da LUT 3:1 para implementar um *multiplexer* 2:1 de acordo com





Selecione uma opçao de resposta:

(I2 I1 I0)	Out
000	0
001	1
010	0
011	1
100	0
101	0
110	1
111	1

tá certa!

(I2 I1 I0)	Out
000	0
001	0
010	0
011	0
100	1
101	1
110	1
111	1

(I2 I1 I0)	Out
000	0
001	1
010	0
011	1
100	0
101	1
110	0
111	1

(I2 I1 I0)	Out
000	1
001	0
010	1
011	0
100	0
101	1
110	0
111	0

Limpar a minha escolha

The sensitivity list of a VHDL process should contain:

Select one:

- the inputs and outputs of the respective process
- the signals and output ports of the respective process
- the ports in which events are triggered by assignments in the process body
- the signals and ports which should trigger the process execution

(enable, inputs) é a lista de sensibilidade do processo
O que significa?
Lista dos sinais/portos dos quais depende o processo (i.e. lista dos sinais/portos cujos eventos afetam os valores calculados pelo processo)

"In which events are triggered" refers to changes in signal values or events that cause a process to execute.
In VHDL, an event is a specific occurrence such as a signal transition (from '0' to '1' or from '1' to '0') or a change in signal value. When an event occurs on a signal or port listed in the sensitivity list of a process, the process is triggered to execute.

An assignment in VHDL refers to the operation of assigning a value to a signal or variable. This assignment can be done using the signal assignment statement (<=) or variable assignment statement (:=).

ARCHITECTURE – Comportamental

Descreve o comportamento do circuito de forma abstrata usando o comando PROCESS.

O Comando PROCESS permite a aplicação de instruções sequenciais e apresenta o formato:

```
PROCESS (<Lista de Sensibilidade>)
BEGIN
    <Descrição Lógica>
END PROCESS;
```

A *lista de sensibilidade* corresponde aos sinais que devem alterar a saída do circuito, e é composta de todos os sinais de entrada para os circuitos combinatórios. Para os registradores assíncronos, a lista seria composta do clock e do reset; e para os registradores síncronos, do clock.

Question 1

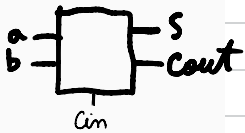
Not yet answered

Marked out of 6.00

Flag question

The synthesis of the following code results in which of the following components:

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity XXXX is
port(   a, b, cin: in std_logic;
        s, cout: out std_logic);
end XXXX;

architecture Behavioral of XXXX is
begin
        cout<=(cin and a) or (cin and b) or (a and b);
        s   <=cin xor a xor b;
end Behavioral;
```



Select one:

- (•) 2 bit Full adder
- ( ) 1 bit Full Subtractor
- ( ) 2 bit Full Subtractor
- ( ) 1 bit Full adder

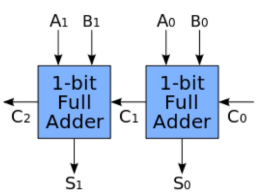
Full adder

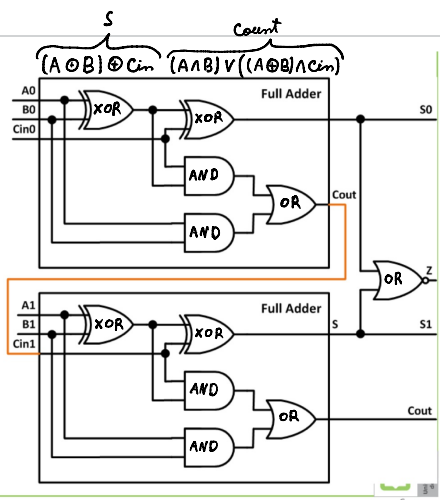
		full adder
0	0	0
0	1	1
1	0	1
1	1	1 0

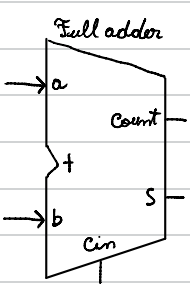
↑ cout

Exemplo : somador de 2 bits com deteção de soma nula (S1=0 e S0=0)

Z = S1' . S0'

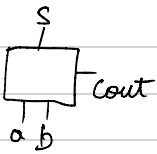


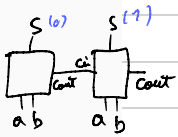




Considering that the FullAdder circuit is a 1-bit full adder, the synthesis of the following

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD.all;

entity Adder is
port(   cin:    in std_logic;
                a,b:  in std_logic_vector(1 downto 0);
                cout: out std_logic;
                s:    out std_logic_vector(1 downto 0));
end Adder;

architecture Structural of Adder is
signal carry: std_logic;
begin
        x: entity work.FullAdder(Behavioral)
                port map (cin => cin,
                          a   => a(0),
                          b   => b(0),
                          cout=> carry,
                          s   => s(0));

        y: entity work.FullAdder(Behavioral)
                port map (cin => carry,
                          a   => a(1),
                          b   => b(1),
                          cout=> cout,
                          s   => s(1));
end Structural;
```





Select one:

- 3 bit Adder
- 2 bit Full adder
- None of the previous
- 4 bits Full adder

Considering the statements in the figure and if A="1001" and B="0011" the result of the operation is

```
signal A, B : unsigned(3 downto 0);
signal Q, R : unsigned(3 downto 0);
...
Q <= A  -  B;
R <= A  /  B;
```

9 3

```
 1001
-0011
------
 0110
```

0-1=1 (vai-1)

Select one:

- Q= "0110" R="0011"
- Q= "1000" R="0011"
- Q= "0110" R="0110"
- Q="0011" R= "0110"

Considering the statements in the figure and if A="0110" and B="0011" the result of the operation is

```
signal A, B : unsigned(3 downto 0);
signal Q, R : unsigned(3 downto 0);
...
Q <= A  +  B;
R <= A rem B;
```

↳ remainder <=> resto

```
 0110
+0011
------
Q=1001
```

Select one:

- Q= "1001" R="0001"
- Q= "1010" R="0000"
- Q= "1101" R="0110"
- Q= "1001" R="0000"

9 a dividir por 3 dá resto 0 por isso R="0000"

Consider the following VHDL code. When A="1111", B="1011" and op="01", the signal R takes the value:

```
entity ALU is
    port(op   : in  std_logic_vector(1 downto 0);
         A, B : in  std_logic_vector(3 downto 0);
         R    : out std_logic_vector(3 downto 0));
end ALU;

architecture Behavioral of ALU is
begin
    R <= std_logic_vector(signed(A) - signed(B)) when op = "00" else
         std_logic_vector(signed(A) + signed(B)) when op = "01" else
         A xor B                                 when op = "10" else
         A or  B;
end Behavioral;
```

Select one:

- ~~"0100"~~
- "1010" $-8+2=-6$
- "1111" $-8+4+2+1=-1$
- "1011" $=-8+2+1=-5$

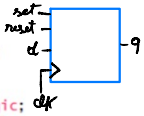
Time left 0:06:31

Question 1
Answer saved
Marked out of 8.00
Flag question

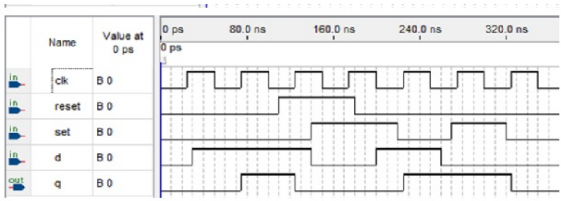
Complete the following VHDL code of the 8 bits ALU circuit. **Do not use spaces in your responses.**

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD.all;

entity ALU4 is

port (a, b : in  std_logic_vector( 7  downto 0);
      op   : in  std_logic_vector( 2  downto 0);
      r, m : out std_logic_vector( 7  downto 0));
end ALU4;

architecture Behavioral of ALU4 is

signal s_a, s_b, s_r :  unsigned ( 7  downto 0);
signal s_m : unsigned ( 15  downto 0);

begin

        s_a <= unsigned(a);
        s_b <= unsigned(b);

        s_m <= s_a * s_b;
        with op select
                s_r <=  s_a + s_b       when "000",
                        s_a - s_b       when "001",
                        s_m( 7  downto 0)    when "010",
                        s_a / s_b       when "011",
                        s_a rem s_b     when "100",
                        s_a and s_b     when "101",
                        s_a or s_b      when "110",
                        s_a xor s_b     when "111";

        r <= std_logic_vector (s_r);
        m <= std_logic_vector (s_m(15 downto  8 )) when (op ="010") else
             (others => '0');
end Behavioral;
```

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.all;
entity SeqElement is
   port(clk : in std_logic;
        d : in std_logic;
        reset : in std_logic;
        set : in std_logic;
        q : out std_logic);
end SeqElement;
architecture Behavioral of SeqElement is
begin
   process( clk )
   begin
      if( rising_edge(clk) ) then
         if (reset = '1') then q <= '0';
         elsif (set = '1') then q <= '1';
         else q <= d;
         end if;
      end if;
   end process;
end Behavioral;
```



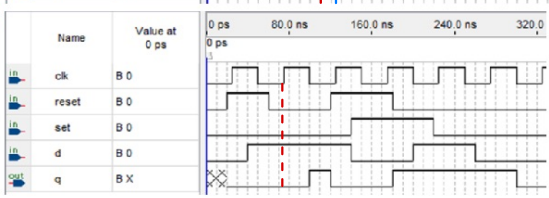
b.



c.

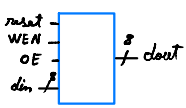
→ el mudanças apenas devia acontecer no rising edge

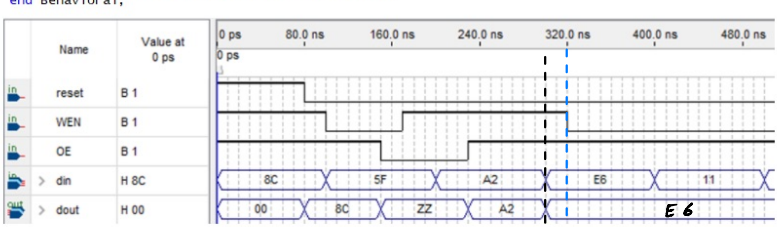
d.



devia estar high

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity LReg is
   port (reset : in std_logic;
         WEN : in std_logic;
         OE  : in std_logic;
         din : in std_logic_vector(7 downto 0);
        dout : out std_logic_vector(7 downto 0));
end LReg;

architecture Behavioral of LReg is
signal s_d: std_logic_vector(7 downto 0);
begin
   process(WEN,reset)
   begin
      if reset = '1' then
         s_d <= (others => '0');
      elsif (WEN = '1') then
            s_d <= din;
      end if;
   end process;
   dout <= (others => 'Z') when OE = '0' else s_d;
end Behavioral;
```





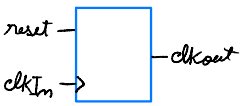
In this simulation for t > 320 ns the output should take the value (in hex notation):

base cases < reset = 1 → 00
OE = 0 → ZZ

- a. 11
- b. 00
- c. Alta-Impedância (3'state)
- d. E6

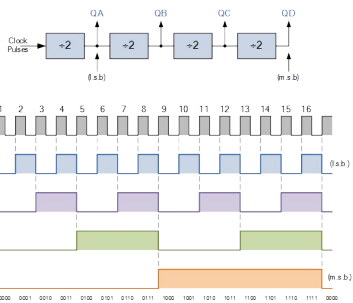
Complete the following VHDL for a divide by 10 frequency divider . **Do not use spaces in your respc**

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.[NUMERIC]_STD.all;
entity FreqDiv is
 port(reset : in std_logic;
      clkIn : in std_logic;
      clkOut : out std_logic);
end FreqDiv;
architecture Behavioral of FreqDiv is
    signal [s_counter] : natural;
    constant k : positive := [4];
begin
 process([clkIn])
 begin
     if rising_edge(clkIn) then
         if ((reset = '1') or (s_counter = [K] - 1)) then
             clkOut <= ['0'];
             s_counter <= [0];
         else
             if (s_counter = [K]/2 - 1) then
                 clkOut <= ['1'];
             end if;
             s_counter <= [s_counter] + 1;
         end if;
    end if;
 end process;
end Behavioral;
```



$$f = \frac{1}{T}$$

??



Consider the VHDL model CntModN based on 4 bit binary up counter (Counters4).

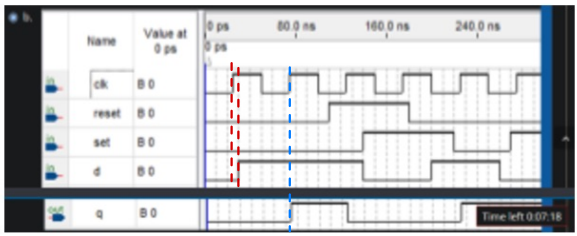
```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD.all;
entity Counters4 is
   port(   clk : in  std_logic;
         reset : in  std_logic;
          load : in std_logic;
        enable : in std_logic;
        dataIn : in std_logic_vector(3 downto 0);
         count : out std_logic_vector(3 downto 0));
   end Counters4;
architecture Behavioral of Counters4 is
signal ...
begin
 ---------------
 ---------------
 ...
end Behavioral;

library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD.all;
entity CntModN is
   port(ck : in  std_logic;
      clear: in  std_logic;
         En : in std_logic;
        cnt : out std_logic_vector(3 downto 0));
   end CntModN;
architecture Struct of CntModN is
signal s_cnt : std_logic_vector(3 downto 0);
signal s_LD  : std_logic;
begin
   Counters: entity work.Counters4(Behavioral)
             port map(clk => ck, reset => clear,
                      enable => En,load => S_LD,
                      datain => (others => '0'),
                      count => s_cnt);

   S_LD  <= s_cnt(0) and s_cnt(3);
   cnt <= s_cnt;
end Struct;
```

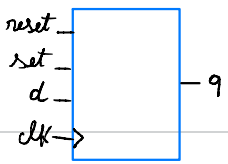
The model CntModN corresponds to a counter modulus:

- a. 9
- b. 16
- c. 12
- d. 10

Clear my choice

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.all;
entity SeqElement is
   port(clk : in std_logic;
        d : in std_logic;
        reset : in std_logic;
        set : in std_logic;
        q : out std_logic);
end SeqElement;
architecture Behavioral of SeqElement is
begin
   process( clk )
   begin
      if( rising_edge(clk) ) then
         if (reset = '1') then q <= '0';
         elsif (set = '1') then q <= '1';
         else q <= d;
         end if;
      end if;
   end process;
end Behavioral;
```





este é muito parecido com um que fiz antes

Time left 0:05:36

Complete the following VHDL code of 4 bit binar y counter with the following counting sequence 0,3,6,9,12. **Do not use spaces in your responses.**

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD.all;
entity CntSpecial is
    port( clk : in std_logic;
        reset : in std_logic;
       enable : in std_logic;
        count : out std_logic_vector( [redacted] downto 0));
 end CntSpecial;
architecture Behavioral of CntSpecial is
[redacted] s_count : unsigned( [redacted] downto 0);

begin
 process( [redacted] )
 begin
     if( rising_edge(clk) ) then
          if (reset = '1') then s_count <= ( [redacted] => '0');
          elsif (enable = '1') then
               if (s_count = [redacted] then
                    s_count <= ( [redacted] => '0');
```

??

s_count

3

s_count

Question 3

Not yet answered

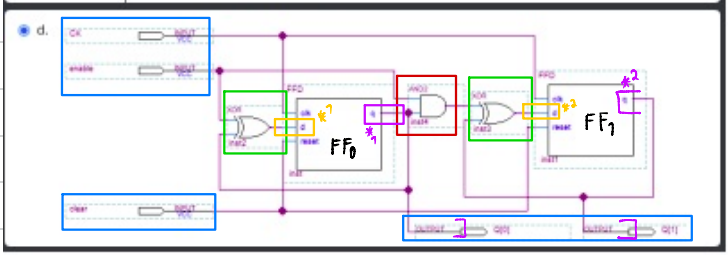
Marked out of 4.0

Flag question

Consider the following VHDL model assuming a pre-existing D Flip-Flop entity.

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
entity TFFS is
   port(ck : in std_logic;
        enable : in std_logic;
        clear : in std_logic;
        qout : out std_logic_vector(1 downto 0));
end TFFS;
architecture Struct of TFFS is
signal S_D : std_logic_vector(1 downto 0);
signal S_Q : std_logic_vector(1 downto 0);
begin
   S_D(0) <= S_Q(0) xor enable;
   S_D(1) <= S_Q(1) xor (S_Q(0) and enable);

   FF0: entity work.FFD(Behavioral)
            port map(clk => ck, reset => clear, d => S_D(0), q => S_Q(0));

   FF1: entity work.FFD(Behavioral)
            port map(clk => ck, reset => clear, d => S_D(1), q => S_Q(1));

   qout <= S_Q;
end Struct;
```

Choose the matching logic circuit.



Question 1

Answer saved

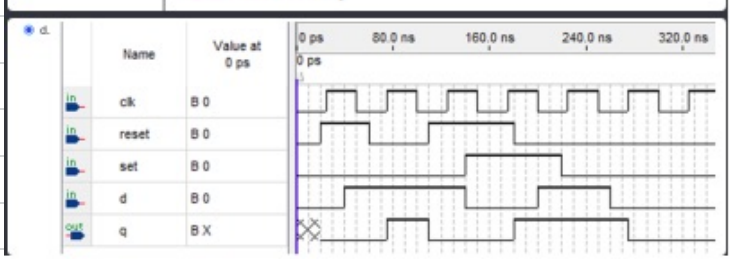
Marked out of 4.0

Flag question

Choose the correct timing diagram for the VHDL model

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
entity SeqElement is
   port(clk : in std_logic;
        d : in std_logic;
        reset : in std_logic;
        set : in std_logic;
        q : out std_logic);
end SeqElement;

architecture Behavioral of SeqElement is
begin
   process(reset, set, clk)
   begin
      if (reset = '1') then q <= '0';
      elsif (set = '1') then q <= '1';
      elsif( rising_edge(clk) ) then q <= d;
      end if;
   end process;
end Behavioral;
```

again some shit gostaram msm desta



The model CntModN corresponds to a counter modulus:

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD.all;
entity Counters4 is
   port(   clk : in  std_logic;
         reset : in  std_logic;
          load : in std_logic;
        enable : in std_logic;
        dataIn : in std_logic_vector(3 downto 0);
         count : out std_logic_vector(3 downto 0));
   end Counters4;
architecture Behavioral of Counters4 is
signal ...
begin
 ---------------
 ---------------
 ...
end Behavioral;

library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD.all;
entity CntModN is
   port(ck : in  std_logic;
      clear: in  std_logic;
         En : in std_logic;
        cnt : out std_logic_vector(3 downto 0));
   end CntModN;
architecture Struct of CntModN is
signal s_cnt : std_logic_vector(3 downto 0);
signal s_LD  : std_logic;
begin
   Counters: entity work.Counters4(Behavioral)
             port map(clk => ck, reset => clear,
                      enable => En,load => S_LD,
                      datain => "0011",
                      count => s_cnt);

   S_LD  <= s_cnt(1) and s_cnt(3);
   cnt <= s_cnt;
end Struct;
```

The model CntModN corresponds to a counter modulus:

- a. 7
- b. 9
- c. 10
- d. 8

Complete the following VHDL code of 4 bit binar y counter with the following counting sequence 0,2,4,6, ...,10.**Do not use spaces in your responses.**

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD.all;
entity CntSpecial is
    port( clk : in std_logic;
        reset : in std_logic;
       enable : in std_logic;
        count : out std_logic_vector( 3 downto 0));
 end CntSpecial;
architecture Behavioral of CntSpecial is
 signal  s_count : unsigned( 3 downto 0);

begin
 process( clk )
 begin
     if( rising_edge(clk) ) then
          if (reset = '1') then s_count <= ( others => '0');
          elsif (enable = '1') then
               if (s_count = "1010" then
                    s_count <= ( others => '0');
               else
                    s_count <= s_count + 3 ;
               end if;
          end if;
     end if

 end process;
    count <= std_logic_vector( s_count );
end Behavioral;
```

Consider the VHDL model of a frequency divider.

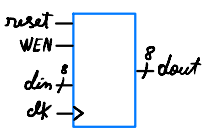
```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD.all;
entity FreqDiv is
   port(reset : in std_logic;
        clkIn : in std_logic;
       clkOut : out std_logic);
end FreqDiv;
architecture Behavioral of FreqDiv is
signal s_counter : natural;
constant K : positive := 10;
begin
   process(clkIn)
   begin
      if rising_edge(clkIn) then
         if ((reset = '1') or (s_counter = K - 1)) then
               clkOut <= '0';
               s_counter <= 0;
         else
            if (s_counter = K/2 - 1) then
               clkOut <= '1';
            end if;
            s_counter <= s_counter + 1;
         end if;
      end if;
   end process;
end Behavioral;
```

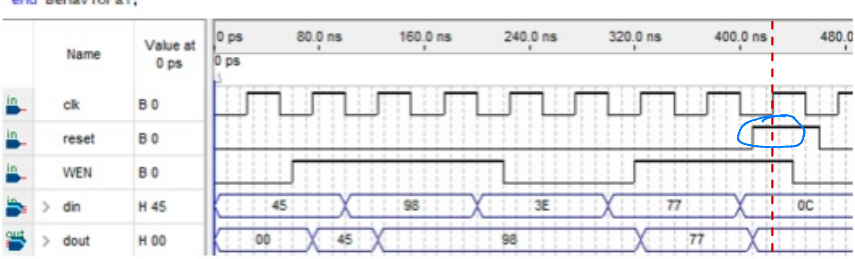
If the input clock period is 20 ns the output clock frequency should be?

- a. 5 MHz
- b. 2 MHz
- c. 500 KHz
- d. 5 KHz

Consider the following VHDL model and a possible timing diagram.

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity Reg is
   port (clk : in std_logic;
         WEN : in std_logic;
         reset : in std_logic;
         din : in std_logic_vector(7 downto 0);
         dout : out std_logic_vector(7 downto 0));
end Reg;

architecture Behavioral of Reg is
signal s_d: std_logic_vector(7 downto 0);
begin
   process(clk,reset)
   begin
      if reset = '1' then
         s_d <= (others => '0');
      elsif rising_edge(clk) then
         if (WEN = '1') then
            s_d <= din;
         end if;
      end if;
   end process;
end Behavioral;
```



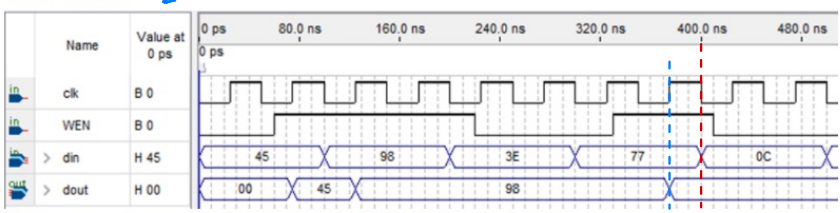


In this simulation for t > 425 ns the output should take the value (in hex notation):

- a. 00
- b. 0C
- c. 98
- d. FF

Consider the following VHDL model and a possible timing diagram.

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity Reg is
   port (clk : in std_logic;
         WEN : in std_logic;
         reset : in std_logic;
         din : in std_logic_vector(7 downto 0);
         dout : out std_logic_vector(7 downto 0));
end Reg;

architecture Behavioral of Reg is
signal s_d: std_logic_vector(7 downto 0);
begin
   process(clk,reset)
   begin
      if reset = '1' then
         s_d <= (others => '0');
      elsif rising_edge(clk) then
         if (WEN = '1') then
            s_d <= din;
         end if;
      end if;
   end process;
end Behavioral;
```



In this simulation for t > 400 ns the output should take the value (in hex notation):

- a. 77
- b. 98
- c. 0C
- d. 00

Complete the following VHDL for a 8 bit register with WRITE ENABLE and OUTPUT ENABLE. **Do not use spaces in your responses.**

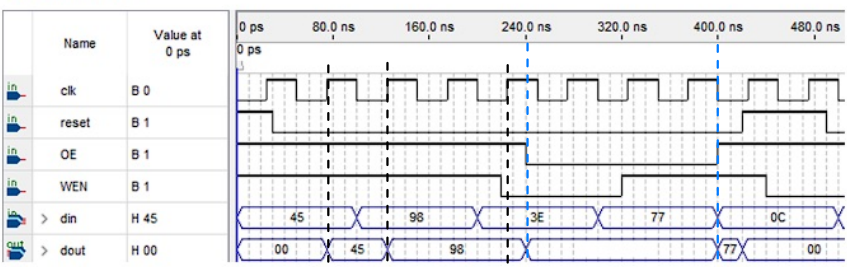
```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity Reg is
     port (clk : in std_logic;
           WEN : in std_logic;
         reset : in std_logic;
            OE : in std_logic;
           din : in std_logic_vector(7 downto 0);
          dout : out std_logic_vector(7 downto 0));
end Reg;

architecture Behavioral of Reg is
  signal  s_d: std_logic_vector (7 downto 0);
begin
     process( clk , reset )
     begin
       if reset = '1' then
          s_d <= (others => '0' );
       elsif rising_edge(clk) then
          if (WEN = '1') then
             s_d <= din ;
          end if;
       end if;
      end process;
       dout  <= (others => '0' ) when OE = '0' else din ;

end Behavioral;
```

Considere o seguinte modelo VHDL e um possível diagrama temporal.

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity Reg is
    port (clk : in std_logic;
          WEN : in std_logic;
          reset : in std_logic;
          OE  : in std_logic;
          din : in std_logic_vector(7 downto 0);
          dout : out std_logic_vector(7 downto 0));
end Reg;

architecture Behavioral of Reg is
signal s_d: std_logic_vector(7 downto 0);
begin
    process(clk,reset)
    begin
        if reset = '1' then
            s_d <= (others => '0');
        elsif rising_edge(clk) then
            if (WEN = '1') then
                s_d <= din;
            end if;
        end if;
    end process;
    dout <= (others => 'Z') when OE = '0' else s_d;
end Behavioral;
```

- Tipo de dados mais utilizado (para representar valores lógicos em sistema digitais)
  - **std_logic** (1 bit) **/ std_logic_vector** (vários bits - barramentos)
    (tipos definidos na biblioteca IEEE.STD_LOGIC_1164)

'0' – Nível baixo	**'1' – Nível Alto**
'L' – Nível baixo passivo	'H' – Nível alto passivo
'Z' – Tri-state (alta impedância)	'-' – Don't care
'X' – Conflito (entre '0' e '1')	'W' – Conflito (entre 'L' e 'H')
'U' – Não inicializado	



Nestas condições de simulação pode dizer-se que para t > 240 ns e t < 400 ns a saída toma o valor em notação hexadecimal de:

- a. 0C
- b. Alta-Impedância (3'state)
- c. 00
- d. 3E

Question 7
Not yet answered
Marked out of 1,0
Flag question

What is the minimum number of 4:1 LUTs required to implement a 4:2 binary priority encoder (with an extra output *idle* to indicate whether the encoding is valid)?

Select one:

- 4
- 1
- 3
- 2

2 + 1 = 3

Analyze the following code. Considering that it is an 8-bit binary counter with synchronous enable and reset, to proceed with the parameterization of the component it is necessary (select only the necessary options):

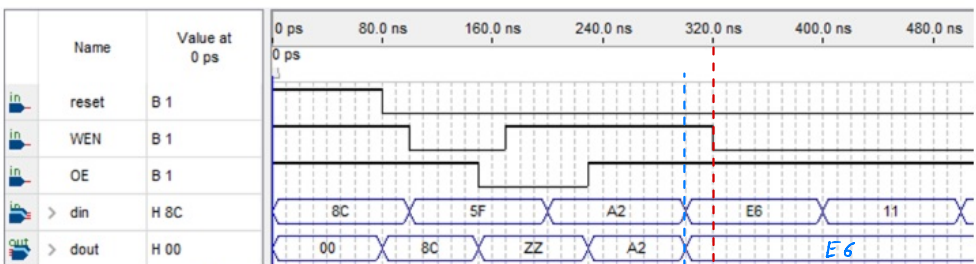
```
entity BinUCntEnRst8 is
  port(reset  : in  std_logic;
       clk    : in  std_logic;
       enable : in  std_logic;
       cntOut : out std_logic_vector(7 downto 0));
end BinUCntEnRst8;
```

Select one or more:

- [ ] Parameterize the cntOut and enable ports
- [ ] Parameterize the reset, enable and cntOut ports
- [ ] Remove the port list and specify it at the top-level ← WTF
- [x] Parameterize the cntOut port

Consider the following VHDL model and a possible timing diagram.

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity LReg is
   port (reset : in std_logic;
         WEN   : in std_logic;
         OE    : in std_logic;
         din   : in std_logic_vector(7 downto 0);
        dout   : out std_logic_vector(7 downto 0));
end LReg;

architecture Behavioral of LReg is
signal s_d: std_logic_vector(7 downto 0);
begin
   process(WEN,reset)
   begin
      if reset = '1' then
         s_d <= (others => '0');
      elsif (WEN = '1') then
            s_d <= din;
      end if;
   end process;
   dout <= (others => 'Z') when OE = '0' else s_d;
end Behavioral;
```



In this simulation for t > 320 ns the output should take the value (in hex notation):

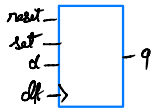
- a. E6
- b. Alta-Impedância (3'state)
- c. 00
- d. 11

Complete the following VHDL for a divide by 10 frequency divider . **Do not use spaces in your response**

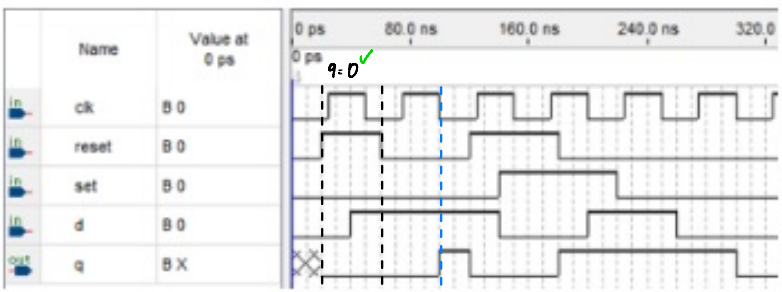
```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.[NUMERIC]_STD.all;
entity FreqDiv is
 port(reset : in std_logic;
      clkIn : in std_logic;
      clkOut : out std_logic);
end FreqDiv;
architecture Behavioral of FreqDiv is
    signal [s_counter] : natural;

    constant k : positive := [10];
begin
 process([clkIn])
 begin
     if rising_edge(clkIn) then
         if ((reset = '1') or (s_counter = [k] - 1)) then

             clkOut <= ['0'];

             s_counter <= ['0'];
         else
             if (s_counter = [k]/2 - 1) then

                 clkOut <= ['1'];

             end if;

             s_counter <= [s_counter] + 1;
         end if;
    end if;
```

Choose the correct timing diagram for the VHDL model

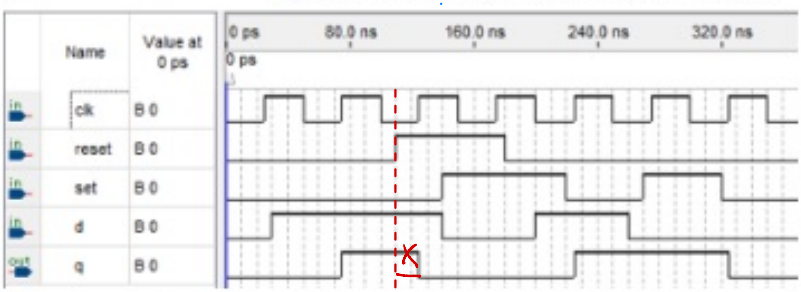
```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
entity SeqElement is
    port(clk : in std_logic;
         d : in std_logic;
         reset : in std_logic;
         set : in std_logic;
         q : out std_logic);
end SeqElement;

architecture Behavioral of SeqElement is
begin
    process(reset, set, clk)
    begin
        if (reset = '1') then q <= '0';
        elsif (set = '1') then q <= '1';
        elsif( falling_edge(clk) ) then q <= d;
        end if;
    end process;
end Behavioral;
```



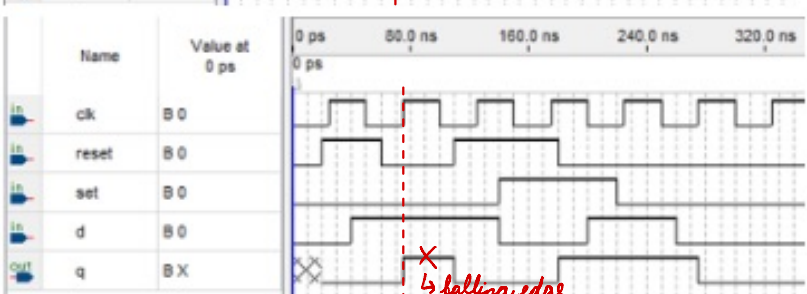
a.



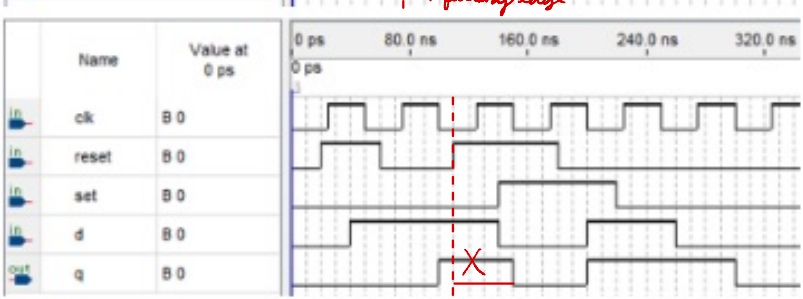
b.



c.



d.



Outros Operadores Aritméticos e Lógicos (ALU de 4 bits)

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD.all;

entity ALU4 is
    port(operation: in  std_logic_vector(2 downto 0);
         operand0 : in  std_logic_vector(3 downto 0);
         operand1 : in  std_logic_vector(3 downto 0);
         result   : out std_logic_vector(3 downto 0);
         multHi   : out std_logic_vector(3 downto 0));
end ALU4;
```

Operation	
000	+
001	-
010	*
011	/
100	rem
101	and
110	or
111	xor

A saída **result** disponibiliza o resultado da operação realizada.

No caso das multiplicações, os 4 bits mais significativos do resultado são disponibilizados em **multHi**.

Complete the following VHDL code of the 8 bits ALU circuit. **Do not use spaces in your respon**

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD.all;

entity ALU4 [ is ]

port (a, b : in  std_logic_vector( [ 7 ] downto 0);
      op   : in  std_logic_vector( [ 2 ] downto 0);
      r, m [ out ] std_logic_vector( [ 7 ] downto 0));
end ALU4;

architecture Behavioral of ALU4 is

signal s_a, s_b, s_r :  unsigned ( [ 7 ] downto 0);
signal s_m : unsigned ( [ 2 ] downto 0);

[ begin ]

        s_a <= unsigned(a);
        s_b <= unsigned(b);
```

Complete the following VHDL code of the 4 bits ALU circuit. **Do not use spaces in your responses.**

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use [ IEEE.NUMERIC_STD.all ] ;

entity ALU4 [ is ]

port (a, b : in  std_logic_vector( [ 3 ] downto 0);
      op   : in  std_logic_vector(2 downto [ 0 ] );
      r, m : [ out ] std_logic_vector(3 downto 0));
end ALU4;

architecture Behavioral of ALU4 is

signal s_a, s_b, s_r :  unsigned (3 downto 0);
signal s_m : unsigned ( [ 7 ] downto 0);

[ begin ]

        s_a <= unsigned(a);
        s_b <= unsigned(b);


        s_m <= s_a * s_b;
        with op select
                s_r <=s_a + s_b         when "000",
                      s_a - s_b                 when "001",
                      s_m( [ 3 ] downto 0)when "010",
                      s_a / s_b                 when "011",
                      s_a rem s_b               when "100",
                      s_a and s_b               when "101",
                      s_a or s_b                when "110",
                      s_a xor s_b               when "111";

        r <= std_logic_vector (s_r);
        m <= std_logic_vector (s_m( [ 7 ] downto [ 4 ] )) when (op ="010") else
             (others => '0');
end Behavioral;
```

```
                                                  when "000",
                                                  when "001",
                                    ownto 0)  when "010",
```

Analyze the following code. To proceed with the parameterization of the component, it is necessary:

```
entity FFD8EnRst is
  port(reset   : in  std_logic;
       clk     : in  std_logic;
       enable  : in  std_logic;
       dataIn  : in  std_logic_vector(7 downto 0);
       dataOut : out std_logic_vector(7 downto 0));
end FFD8EnRst;
```

tá certo!

Select one or more:

- [x] Parameterize the dataIn and dataOut ports
- [ ] Parameterize the reset, clk, enable, dataIn and dataOut ports
- [ ] Add a new port
- [ ] Remove the port list and specify it at the top-level

Pergunta **5**

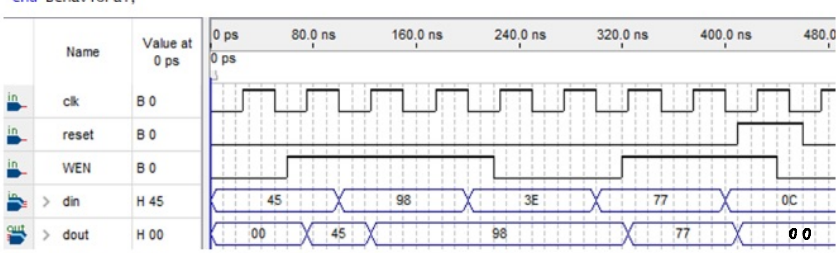
Por responder

Nota: 2,0

Marcar pergunta

Considere o seguinte modelo VHDL e um possível diagrama temporal.

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity Reg is
   port (clk : in std_logic;
         WEN : in std_logic;
         reset : in std_logic;
         din : in std_logic_vector(7 downto 0);
         dout : out std_logic_vector(7 downto 0));
end Reg;

architecture Behavioral of Reg is
signal s_d: std_logic_vector(7 downto 0);
begin
   process(clk,reset)
   begin
      if reset = '1' then
         s_d <= (others => '0');
      elsif rising_edge(clk) then
         if (WEN = '1') then
            s_d <= din;
         end if;
      end if;
   end process;
end Behavioral;
```



Nestas condições de simulação pode dizer-se que para t > 425 ns a saída toma o valor em notação hexadecimal de:

- a. 0C
- b. 00
- c. 98
- d. FF